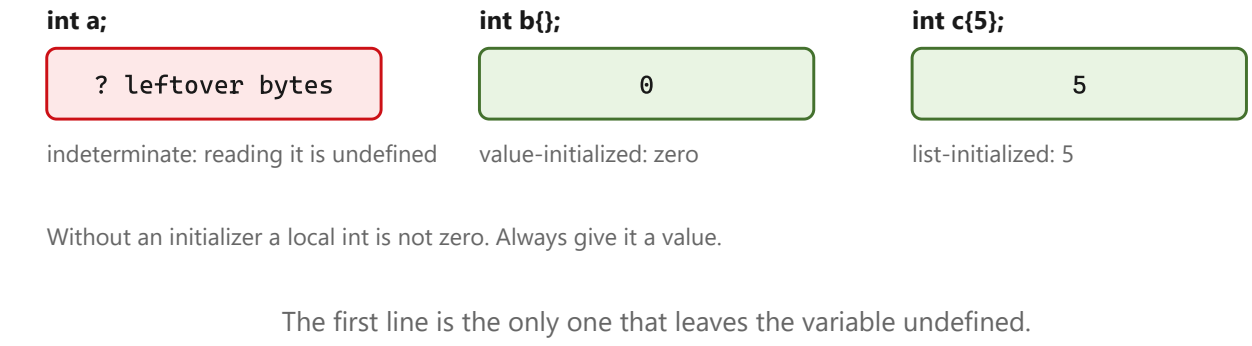
Types, Initialization and Constants

Initialization forms and brace traps, `auto` and `decltype`, `const`, `constexpr` and `static`, and what C++14 to 20 added. Casts and Type Conversions is the next lesson.

Initialization and brace traps

Every variable should start with a value. C++ has several syntaxes for it, and they differ in what they leave behind and in what they refuse to convert.




```
// default-initialized: INDETERMINATE value, never read it
int a;
int b{};          // value-initialized: zero
int c = 5;        // copy-initialized
int d(5);         // direct-initialized
// list-initialized: the preferred form, rejects narrowing
int e{5};
// int f{3.7};    // ERROR: narrowing double → int inside braces
int g = 3.7;      // compiles: silently truncates to 3
```

KIND	EXAMPLE	NOTES
aggregate	`Point p{1, 2};`	no constructors: members set in order
designated (C++20)	`Point p{.x = 3, .y = 4};`	names the members, order must match
value-initialize	`Point p{};`	every member zero

BRACES PREFER `INITIALIZER_LIST`

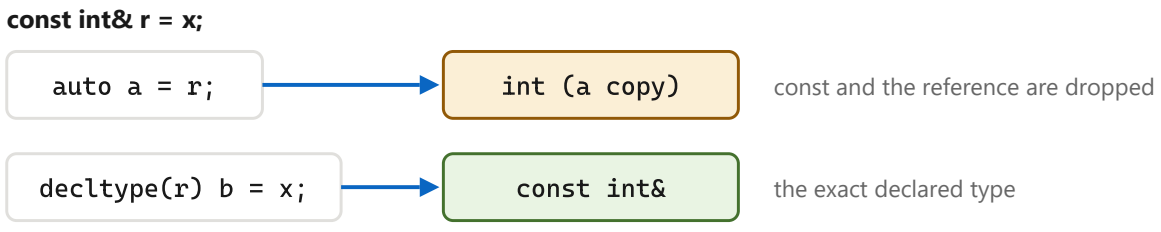
```cpp
std::vector<int> v1(3, 7);   // parentheses: 3 elements, each 7
// braces: the initializer_list constructor wins → the two elements
// 3 and 7
std::vector<int> v2{3, 7};
```

If a class has an `initializer_list` constructor, `{ ... }` picks it even when another constructor matches better (compiled: `Widget w{1, 2}` called the list constructor, `Widget w(1, 2)` the (int, int) one). Empty braces `{}` call the default constructor.

- **Initialize everything.** An uninitialized local is **indeterminate**; reading it is undefined behaviour.
- Braces catch accidental narrowing at compile time: a free safety check.
- `int{x}` is list-initialization, not a `static_cast`: it converts, but must not narrow, so the compiler flags a conversion that could lose information.

`auto`, `decltype` and modern syntax

`auto` lets the compiler write the type from the initializer, and `decltype` asks for the exact type of an expression. They agree less often than you would expect.



auto copies by default; decltype keeps everything.

```cpp
int x = 0;
const int& r = x;
// int auto drops const and the reference
auto a = r;
// const int& decltype gives the exact declared type
decltype(r) b = x;
// const int& decltype(auto) = "deduce like auto, but keep it exact"
decltype(auto) c = r;

auto t = {1, 2, 3};       // std::initializer_list<int>, NOT a vector
```

AUTO TRAPS

`auto y = v[0];` *copies*: write `const auto&` to avoid it. Same for `for (auto [k, v] : map)` (a copy of each pair). `auto` hides what a function returns: fine for iterators and lambdas, poor for public interfaces.

FEATURE	INSTEAD OF	EXAMPLE
Range-based `for`	index / iterator loops	`for (const auto& x : v) {}`
`nullptr`	`NULL` / `0`	`int* p = nullptr;`
`using` alias	`typedef`	`using Map = std::map<int, std::string>;`
`override` / `final`	hoping you matched the signature	`void f() override;`
`= default` / `= delete`	empty bodies, private unimplemented copy	`Foo(const Foo&) = delete;`
`static_assert`	runtime checks of compile-time facts	`static_assert(sizeof(int) == 4);`
Structured bindings (C++17)	`.first` / `.second`	`for (auto& [key, val] : m) {}`
`if` with initializer (C++17)	variable leaking into outer scope	`if (auto it = m.find(k); it ≠ m.end()) {}`
`[[nodiscard]]`	ignored error codes	`[[nodiscard]] bool open();`

`const`, `constexpr` and `static`

- `const`: unchangeable after initialization. Use everywhere.
- `constexpr`: computable **at compile time** (array sizes, template arguments), and still callable at runtime.

```
constexpr int square(int x) { return x * x; }
constexpr int N = square(4);          // 16, computed by the compiler
// size 9: legal because it is a constant expression
std::array<int, square(3)> a;
int runtime = readInput();
// also fine: same function runs at runtime
int r = square(runtime);

static_assert(N == 16, "math is broken");   // checked at compile time
```

`static` means three things:

WHERE	MEANING
Local variable in a function	one instance that lives for the whole program and keeps its value between calls
Global / namespace scope	internal linkage: visible only inside this `.cpp` file
Class member	belongs to the class, shared by all objects

Linkage and `extern "C"`

C++ **mangles** function names (it encodes the parameter types so that overloads get different symbols); C does not. To call C code, or to be called by it, tell the compiler to use C linkage:

```
// an unmangled symbol, callable from C
extern "C" int compareInts(const void* a, const void* b);

// a C API taking a callback (compiled: sorted 12345)
std::qsort(arr, 5, sizeof(int), compareInts);

// a whole header of C declarations
extern "C" {
    #include "legacy.h"
}
```

- An `extern "C"` function cannot be overloaded and cannot be a template or a member.
- Pointers to `extern "C"` functions are technically a different type from C++ function pointers; compilers accept them for C callbacks in practice.
- **Linkage rules:** a non-`static`, non-`inline` function defined in a header and included twice gives a "multiple definition" link error; mark it `inline`. `extern` on a variable declares it without defining it.

What C++14, 17 and 20 added

A map of which standard added what, so you know which flag a feature needs:

STANDARD	FEATURE	ONE-LINER
C++11	move semantics, smart pointers, lambdas, `auto`, `constexpr`, `std::thread`, variadic templates, `nullptr`, range-for	The big modernisation.
C++14	generic lambdas, `make_unique`, return type deduction, binary literals `0b101`	Polishes C++11.
C++17	`std::optional`, `std::variant`, `std::string_view`, structured bindings, `if constexpr`, fold expressions, `std::filesystem`, parallel algorithms, `std::scoped_lock`, CTAD	Vocabulary types and compile-time branching.
C++20	concepts, ranges, `std::span`, `operator⟺`, coroutines, modules, `std::jthread`, `std::format`	Constraints on templates, composable algorithms.

The C++17 and C++20 vocabulary types (`optional`, `variant`, `span`, `string_view`) have their own lesson: Vocabulary Types.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Initialization and brace traps	C++ Primer	2.2 Variables	70 to 78
	Programming: Principles and Practice	8.4 to 8.6 Evolving a class (Date), enumerations, operator overloading	238 to 249
auto, decltype and modern syntax	C++ Primer	2.5 Dealing with types	96 to 100
	Programming: Principles and Practice	2.8 to 2.10 Type safety, conversions, type deduction with auto	56 to 60
	Effective Modern C++	Item 2, Understand auto type deduction	36 to 41
	Effective Modern C++	Item 5, Prefer auto to explicit type declarations	55 to 61
const, constexpr and static	C++ Primer	6.5 Features for specialized uses	265 to 270
	Effective Modern C++	Item 15, Use constexpr whenever possible	115 to 121